

Ingeniería de Software

Cómo se concibe, desarrolla, integra, despliega, opera y evoluciona el software.

Cómo nace un software

Un producto de software normalmente evoluciona desde una necesidad hacia requisitos, diseño de la solución, desarrollo, pruebas, entrega, operación y mantenimiento. QA puede contribuir desde el análisis del requisito hasta la observación en producción.

SDLC y STLC

SDLC significa Software Development Life Cycle. STLC significa Software Testing Life Cycle. El primero describe el ciclo de desarrollo; el segundo organiza actividades de prueba como análisis, planificación, diseño, preparación, ejecución y cierre.

Requisitos funcionales y no funcionales

Los requisitos funcionales describen lo que el sistema debe hacer. Los no funcionales cubren atributos como rendimiento, seguridad, accesibilidad, confiabilidad y usabilidad. QA debe cuestionar ambigüedades y convertir requisitos en condiciones verificables.

Arquitecturas que encuentra QA

Monolitos, arquitecturas por capas, cliente-servidor, microservicios, sistemas orientados a eventos y serverless cambian los lugares donde pueden surgir fallas. Comprender componentes y dependencias mejora la investigación.

Frontend, backend e integraciones

Frontend es la capa de interacción con el usuario; backend ejecuta reglas y procesamiento; APIs, colas y eventos conectan servicios. Un defecto visible en pantalla puede originarse en cualquiera de estas capas.

Calidad arquitectónica

Rendimiento, seguridad, confiabilidad, mantenibilidad, compatibilidad y escalabilidad deben considerarse desde el diseño. Testabilidad y observabilidad también influyen en la capacidad de detectar y diagnosticar problemas.

Deuda técnica, refactoring y mantenimiento

La deuda técnica representa decisiones que aceleran una entrega pero pueden aumentar el costo futuro. El refactoring mejora la estructura interna sin intención de cambiar el comportamiento; aun así puede requerir regresión basada en riesgo.

Visión avanzada para QA

Un QA experimentado no necesita ser arquitecto, pero debe poder leer flujos, identificar puntos de falla y discutir riesgos, dependencias, contratos, datos y mecanismos de recuperación.

Consejo de QA

Al investigar un problema, dibuje el flujo de datos antes de asumir que la causa está en la interfaz.